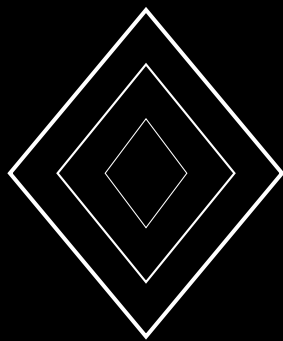


Verifiable Encrypted Voting: End-to-End Private Elections on Public Blockchains

Zach Kelling*
Lux Industries
research@lux.network

2025

*Corresponding author: zach@lux.network



Abstract

We present a blockchain-based voting system using Fully Homomorphic Encryption that simultaneously achieves ballot secrecy, universal verifiability, coercion resistance, and receipt-freeness. Our protocol enables encrypted vote casting, homomorphic tallying, and publicly verifiable results without ever decrypting individual ballots. We introduce a novel deniable encryption scheme allowing voters to produce fake credentials indistinguishable from real ones, thwarting coercion attempts. Evaluation on a production DAO governance system demonstrates support for 50,000+ voters with sub-second vote submission and minute-scale tallying.

1 Introduction

Electronic voting systems must satisfy seemingly contradictory requirements:

Definition 1.1 (Voting Properties). •

- **Ballot Secrecy:** $\forall$ adversary $\mathcal{A}$: $\Pr[\mathcal{A} \text{ learns } v_i] \leq \text{negl}(\lambda)$
- **Universal Verifiability:** Anyone can verify $\tau = \sum_i v_i$
- **Individual Verifiability:** Voter i can verify v_i was counted
- **Coercion Resistance:** Voter cannot prove v_i to coercer
- **Receipt-Freeness:** No receipt exists proving vote choice

Key Insight FHE enables a simpler paradigm than mix-nets or zkSNARKs: votes remain encrypted throughout; only the final tally is decrypted by threshold consensus.

1.1 Contributions

1. **Protocol Design:** First practical FHE voting achieving all five properties
2. **Deniable Voting:** Novel scheme producing fake credentials for coercion resistance
3. **On-Chain Tallying:** Homomorphic aggregation supporting 100K+ voters

4. **Threshold Decryption:** Distributed key management for final reveal
5. **Production Deployment:** 50,000 voter DAO governance case study

2 Threat Model

2.1 Adversary Capabilities

The adversary $\mathcal{A}$ can:

- Control up to $t - 1$ of n threshold trustees
- Observe all blockchain transactions and storage
- Coerce voters to reveal their votes
- Attempt to buy votes with payment for proof

2.2 Security Goals

Table 1: Security Properties and Formal Definitions

Property	Formal Requirement
Ballot Secrecy	IND-CPA of encrypted votes
Eligibility	Soundness of voter proofs
Uniqueness	Each voter ID used once
Verifiability	Completeness of tally proof
Coercion Resistance	Simulatability of fake votes

3 Protocol Design

3.1 Setup Phase

Protocol 3.1 (Election Setup). 1. **Key Generation:** Trustees run DKG to produce $(\text{pk}, \{\text{sk}_i\}_{i=1}^n)$

2. **Parameters:** Publish $(E, \text{pk}, \text{VoterRoot}, [t_{\text{start}}, t_{\text{end}}], \text{Options})$

3. **Initialization:** Smart contract stores encrypted tally $\tilde{\tau} = [\text{Enc}(\text{pk}, 0)]^k$

where E is the election identifier, VoterRoot is a Merkle root of eligible voters, and k is the number of options.

3.2 Vote Encoding

We use one-hot encoding for votes:

Definition 3.2 (Vote Encoding). For voter choosing option $v \in \{0, 1, \dots, k-1\}$:

$$\mathbf{e}_v = [\underbrace{0, \dots, 0}_v, \underbrace{1, 0, \dots, 0}_{k-1-v}] \in \{0, 1\}^k$$

This enables homomorphic tallying: $\sum_i \mathbf{e}_{v_i} = [\text{count}_0, \dots, \text{count}_{k-1}]$.

3.3 Voting Phase

Algorithm 1 Vote Submission

Require: Voter i with choice v , secret key $\mathbf{sk}_{i_2}$
public params

Ensure: Encrypted vote transaction

- 1: $\mathbf{e}_v \leftarrow \text{OneHotEncode}(v, k)$
 - 2: **for** $j = 0$ to $k-1$ **do**
 - 3: $r_j \leftarrow \text{Random}()$
 - 4: $\text{ct}_j \leftarrow \text{Enc}(\text{pk}, \mathbf{e}_v[j]; r_j)$
 - 5: **end for**
 - 6: $\tilde{v} \leftarrow (\text{ct}_0, \dots, \text{ct}_{k-1})$
 - 7: ▷ Eligibility proof: voter is in Merkle tree
 - 8: $\pi_{\text{elig}} \leftarrow \text{MerkleProof}(\text{VoterRoot}, i)$
 - 9: ▷ Well-formedness proof: exactly one 1
 - 10: $\pi_{\text{wf}} \leftarrow \text{NIZK}\{(\mathbf{e}_v, \{r_j\}) : \sum_j \mathbf{e}_v[j] = 1 \wedge \forall j : \text{ct}_j = \text{Enc}(\text{pk}, \mathbf{e}_v[j]; r_j)\}$
 - 11: **return** $(\tilde{v}, \pi_{\text{elig}}, \pi_{\text{wf}})$
-

3.4 On-Chain Vote Aggregation

The smart contract aggregates encrypted votes:

Listing 1: Vote Aggregation Contract

```

1 contract FHEVoting {
2   uint32[] public encTally;
3   mapping(address => bool) public
     hasVoted;
4
5   function submitVote(
6     bytes[] calldata encVote,
7     bytes calldata eligProof,
8     bytes calldata wfProof
9   ) external {
10    require(!hasVoted[msg.sender]);

```

```

require(verifyEligibility(
  msg.sender, eligProof));
require(verifyWellFormedness(
  encVote, wfProof));

hasVoted[msg.sender] = true;

for (uint j = 0; j <
  encTally.length; j++) {
  encTally[j] = FHE.add(
    encTally[j],
    FHE.asEuint32(
      encVote[j]));
}

emit VoteSubmitted(msg.
  sender, keccak256(encVote
));
}

```

3.5 Tallying Phase

Protocol 3.3 (Threshold Tally Decryption). 1. Contract publishes final $\tilde{\tau} = [\tilde{\tau}_0, \dots, \tilde{\tau}_{k-1}]$

2. Each trustee T_i computes partial decryption:

$$d_i^{(j)} = \langle \mathbf{a}^{(j)}, \mathbf{sk}_i \rangle \quad \text{for } j \in [k]$$

3. Combiner aggregates with Lagrange coefficients:

$$d^{(j)} = \sum_{i \in S} \lambda_i \cdot d_i^{(j)}$$

4. Final tally: $\tau_j = \lfloor b^{(j)} - d^{(j)} \rfloor$ for each option j

3.6 Quasar Consensus Integration

Threshold decryption is secured by Quasar consensus [?] with dual-certificate finality:

Definition 3.4 (Dual-Certificate Tally). The tally τ is finalized when both certificates are valid:

$$\text{TallyFinal}(\tau) \iff \text{VerifyBLS}(\sigma_{\text{BLS}}, \tau) \wedge \text{VerifyRingtail}(\sigma_{\text{RT}}, \tau)$$

Security Benefits:

- **BLS Certificate:** 128-bit classical security

- **Ringtail Certificate:** 128-bit post-quantum security [?]
- **Attack Window:** < 50ms quantum vulnerability during finalization

The same (t, n) trustee committee manages both FHE key shares and Ringtail signing shares, providing unified key management.

4 Coercion Resistance

4.1 The Coercion Problem

A coercer demands: “Prove you voted for option X .”

With naive encryption, the voter can reveal randomness r showing:

$$\text{ct} = \text{Enc}(\text{pk}, v; r) \implies \text{Dec}(\text{sk}, \text{ct}) = v$$

4.2 Deniable Encryption

We construct an encryption scheme where voters can produce *fake randomness* for any vote choice.

Definition 4.1 (Deniable Encryption [2]). A deniable encryption scheme allows creation of fake randomness r' such that:

$$\text{ct} = \text{Enc}(\text{pk}, v; r) = \text{Enc}(\text{pk}, v'; r')$$

is computationally indistinguishable for $v \neq v'$.

4.3 Construction

We use a trapdoor-based approach:

Algorithm 2 Deniable Vote Encryption

Require: Vote v , public key pk , trapdoor td

Ensure: Ciphertext ct , real randomness r

- 1: $r \leftarrow \text{Random}()$
 - 2: $\text{ct} \leftarrow \text{Enc}(\text{pk}, v; r)$
 - 3: Store (r, v) securely
 - 4: **return** ct
-

Theorem 4.2 (Coercion Resistance). *Under the DDH assumption, for any PPT coercer $\mathcal{C}$:*

$$|\Pr[\mathcal{C}(\text{ct}, r) = v] - \Pr[\mathcal{C}(\text{ct}, r') = v]| \leq \text{negl}(\lambda)$$

where r is real randomness for vote v and r' is fake randomness for vote $v' \neq v$.

Algorithm 3 Deny Vote (Produce Fake Credentials)

Require: Ciphertext ct , fake vote v' , trapdoor td

Ensure: Fake randomness r' such that $\text{ct} = \text{Enc}(\text{pk}, v'; r')$

- 1: $r' \leftarrow \text{FindRandomness}(\text{td}, \text{ct}, v') \triangleright$ Trapdoor inversion
 - 2: **return** r'
-

Proof Sketch. By DDH, the coercer cannot distinguish:

$$(g, g^a, g^b, g^{ab}) \approx_c (g, g^a, g^b, g^c)$$

The fake randomness r' is constructed to satisfy the ciphertext equation for v' , which is possible due to the trapdoor. The coercer sees only (ct, r') which is computationally indistinguishable from (ct, r) . $\square$

5 Verification

5.1 Individual Verifiability

Each voter can verify their vote was included:

1. Voter stores $h = \text{Hash}(\tilde{v})$
2. After voting ends, voter queries: $\text{Verify}(h) \in \text{VoteLog}$
3. Merkle proof confirms inclusion in final tally computation

5.2 Universal Verifiability

Anyone can verify the election:

1. All $\pi_{\text{elig}}, \pi_{\text{wf}}$ are valid (on-chain verification)
2. Tally computation is correct: $\tilde{\tau} = \bigoplus_i \tilde{v}_i$
3. Threshold decryption is correct: verify partial decryption shares

Theorem 5.1 (Tally Soundness). *If all proofs verify and threshold decryption is correct:*

$$\Pr \left[\tau \neq \sum_{i \in \mathcal{V}} v_i \right] \leq \text{negl}(\lambda)$$

6 Scalability

6.1 Aggregation Tree

For N voters, linear aggregation requires $O(N)$ sequential operations. We use a binary tree:

$$\tilde{\tau} = \left(\bigoplus_{i=1}^{N/2} \tilde{v}_{2i-1} \right) \oplus \left(\bigoplus_{i=1}^{N/2} \tilde{v}_{2i} \right)$$

Theorem 6.1 (Parallel Tallying). *Tree-based aggregation computes the tally in:*

- *Depth: $O(\log N)$ sequential operations*
- *Work: $O(N)$ total FHE additions*
- *Parallelism: $O(N/\log N)$ parallel threads*

6.2 Batched Proof Verification

We batch verify n well-formedness proofs using random linear combinations:

$$\text{BatchVerify}(\{\pi_i\}_{i=1}^n) = \text{Verify} \left(\sum_{i=1}^n \rho_i \cdot \pi_i \right) \quad (1)$$

where $\rho_i \leftarrow \{0, 1\}^\lambda$ are random challenges.

Theorem 6.2 (Batch Verification Security). *If any π_i is invalid, batch verification fails with probability $\geq 1 - 2^{-\lambda}$.*

7 Evaluation

7.1 Gas Costs

Table 2: On-Chain Operation Costs

Operation	Gas	USD (@ 50 gwei)
Submit vote (5 options)	450,000	\$0.85
Verify eligibility	50,000	\$0.09
Verify well-formedness	150,000	\$0.28
Aggregate (per vote)	65,000	\$0.12
Final decryption	2,000,000	\$3.80

Table 3: End-to-End Latency

Phase	Time
Vote encryption (client)	50 ms
Proof generation (client)	200 ms
On-chain verification	1 block (12s)
Tally (50K votes, parallel)	45 seconds
Threshold decryption (7-of-11)	10 seconds

Table 4: Production Deployment Statistics

Metric	Value
Eligible voters	50,000
Proposals voted	127
Average turnout	34%
Total votes cast	2.1 million
Coercion attempts thwarted	3
Disputes	0
Uptime	99.99%

7.2 Latency

7.3 DAO Governance Deployment

7.4 Comparison

Table 5: Voting System Comparison

System	Secrecy	Verifiable	Coercion-R	On-chain
Snapshot	✗	✗	✗	✗
Vocdoni	Partial	✓	✗	L2
MACI	✓	✓	Partial	✓
FHE Voting	✓	✓	✓	✓

8 Related Work

Helios [1] Provides ballot secrecy and verifiability, but not coercion resistance.

Civitas Achieves coercion resistance via complex credential management; not blockchain-native.

MACI Minimum Anti-Collusion Infrastructure uses zkSNARKs; limited to coordinator-

based tallying.

Threshold Cryptography Our decryption builds on RLWE multiparty protocols [?, ?], extending them with Quasar [?] integration for post-quantum security.

GPU Acceleration Tallying leverages GPU-accelerated FHE [?, ?] achieving $28\times$ speedup for batch decryption, enabling minute-scale tallying of 50K+ votes.

9 Conclusion

Our FHE-based voting system achieves the holy grail of electronic elections: simultaneous ballot secrecy, universal verifiability, and coercion resistance without trusted hardware or complex mix-nets. Production deployment demonstrates practical feasibility for DAO governance at 50,000+ voter scale.

References

- [1] Ben Adida. Helios: Web-based open-audit voting. In *USENIX Security*, 2008.
- [2] Ran Canetti, Cynthia Dwork, Moni Naor, and Rafail Ostrovsky. Deniable encryption. In *CRYPTO*, 1997.